



School of Future Tech

Case Study Report

on

SystemFitnessMembershipSystem

by

Name:Arnesh Munj

Roll no:150096725034

About me: B.Tech 2025-2029 JAVA Project

Index

1. Introduction to the Case Study.
2. Problem Statement / Case Background (Abstract).
3. Problem Statement / Case Study Design.
4. Methods & Algorithms Technology Applied in the Problem Statement / Case Study.
5. Problem Statement / Case Study Implementation Details and Snapshots.

6. Problem Statement / Case Study Results and Conclusion.
7. References

1. Introduction to the Case Study

Fitness centers handle a steady flow of new members, each choosing a membership plan, a duration and sometimes qualifying for a discount. Doing this on paper is slow and error-prone, especially when the fee depends on several factors at once.

This case study builds a console-based Fitness Membership Management System in Java. It registers members, shows the available plans, calculates the fee, applies discounts and prints a membership summary. The aim is to apply core Java concepts (Scanner input, switch, if-else, operators, methods and loops) to a realistic problem.

2. Problem Statement / Case Background (Abstract)

A fitness center requires a system to register members, select membership plans, calculate fees and determine membership duration.

Objectives

- Register members.
- Display membership plans.
- Calculate membership fees.
- Apply discounts.
- Generate membership details.

Required Java implementation

- Scanner to capture member information.
- A switch statement to select membership plans.
- If-Else statements to apply discounts based on duration or category.
- Operators to calculate fees.
- Methods to handle fee calculations.
- Loops to process multiple member registrations.

Abstract: The developed program takes a member's name, age, category, chosen plan and duration. It computes the total fee, applies a combined duration and category discount, prints a summary, and repeats for as many members as the user wants. At the end it lists all registered members and the total collection.

3. Case Study Design

3.1 Modules

No.	Module	Implemented by
-----	--------	----------------

1	Member Registration	Input section in <code>main()</code>
2	Plan Selection	<code>displayPlans()</code> , <code>getPlanName()</code> , <code>getPlanRate()</code>
3	Fee Calculation	<code>calculateFee()</code>
4	Discount Application	<code>getDiscountPercent()</code> , <code>applyDiscount()</code>
5	Membership Summary	<code>printSummary()</code> and the final report loop

3.2 Membership Plans

Choice	Plan	Rate per month
1	Basic	Rs. 1000
2	Standard	Rs. 1500
3	Premium	Rs. 2500

3.3 Discount Rules

Type	Condition	Discount
Duration	3 to 5 months	5%
Duration	6 to 11 months	10%
Duration	12 months or more	20%
Category	Student	additional 10%
Category	Senior Citizen	additional 15%
Category	Regular	none

The duration discount and the category discount are added together to give the total discount percentage.

3.4 Program Flow

1. Start the program and show the welcome message.
2. Read the member's name, age and category.
3. Display the plans and read the plan choice (repeat until valid).
4. Read the duration in months (repeat until greater than zero).

5. Calculate the fee, the discount percentage and the final fee.
6. Print the membership summary and store the member's name and final fee in lists.
7. Ask whether to register another member; if yes, go to step 2.
8. Print the list of all members, the member count and the total collection.

4. Methods & Algorithms Technology Applied in the Problem Statement / Case Study

4.1 Technology

- Language: Java
- Classes used: `Scanner` for input and `ArrayList` for storing members
- Interface: console (command line)

4.2 Java Concepts Applied

- **Scanner:** reads the name, age, category, plan choice, duration and the repeat choice from the keyboard.
- **switch statement:** `getPlanName()` and `getPlanRate()` map the plan number to its name and monthly rate. A `default` case handles invalid values.
- **if-else statements:** `getDiscountPercent()` checks the duration range first and then the category to build the discount.
- **Operators:** arithmetic operators (`*`, `/`, `-`, `+`), relational operators (`>=`, `<=`, `<`, `>`) and logical operators (`|`, `&`) are used for fees, discounts and input validation.
- **Methods:** each task has its own method, which keeps `main()` short and the code easy to modify.
- **Loops:** a `do-while` loop handles multiple registrations and input validation, and a `for` loop prints the final member list.
- **ArrayList:** two lists store member names and final fees, so the total collection can be computed at the end.

5. Problem Statement / Case Study Implementation Details

5.1 Implementation Details

The whole program is in one file, `FitnessMembershipSystem.java`, inside the class `FitnessMembershipSystem`. All helper methods are static so they can be called directly from `main()`. A single `Scanner` object is declared at class level and shared by all input statements.

Key methods:

- `displayPlans()` prints the plan menu.
- `getPlanName(int choice)` and `getPlanRate(int choice)` use a switch statement to return the plan name and the monthly rate.
- `getCategoryName(int category)` returns the category text using if-else.
- `calculateFee(int rate, int months)` returns $\text{rate} * \text{months}$.
- `getDiscountPercent(int months, int category)` returns the combined discount percentage.
- `applyDiscount(double fee, double discountPercent)` returns the fee after the discount.
- `printSummary(...)` prints the membership details.

The key calculation in `main()`:

```
double fee = calculateFee(rate, months);
```

```
double discountPercent = getDiscountPercent(months, category);
```

```
double finalFee = applyDiscount(fee, discountPercent);
```

```

/opt/homebrew/opt/java/libexec/openjdk.jdk/Contents/Home/bin/java -javaagent:/Applications/IntelliJ IDEA.app/Contents/lib/idea_rt.jar=58700 -Dfile.encoding=UTF-8 -Dsun.stdout.encoding=UTF-8
===== Fitness Membership Management System =====

Enter member name: Rahul Sharma
Enter age: 20
Select category:
1. Regular
2. Student
3. Senior Citizen
Enter choice: 2

----- Membership Plans -----
1. Basic - Rs. 1000 per month
2. Standard - Rs. 1500 per month
3. Premium - Rs. 2500 per month
Select plan (1-3): 2
Enter duration in months: 12

===== Membership Summary =====
Member Name : Rahul Sharma
Age : 20
Category : Student
Plan : Standard
Duration : 12 month(s)
Total Fee : Rs. 18000.0
Discount : 30.0%
Final Fee : Rs. 12600.0
=====

Register another member? (y/n):
  
```

6. Problem Statement / Case Study Results and Conclusion.

6.1 Test Cases and Results

Member	Category	Plan	Months	Fee (Rs.)	Discount	Final Fee (Rs.)
Rahul Sharma	Student	Standard	12	18000	20% + 10% = 30%	12600
Meena Iyer	Senior Citizen	Premium	6	15000	10% + 15% = 25%	11250

Total collection for the two members is Rs. 23850. Both results match the manual calculation:

- Rahul: $1500 \times 12 = 18000$; 30% of 18000 = 5400; $18000 - 5400 = 12600$.
- Meena: $2500 \times 6 = 15000$; 25% of 15000 = 3750; $15000 - 3750 = 11250$.

Invalid plan numbers and non-positive durations are rejected and the user is asked again.

6.2 Conclusion

The system meets every objective of the case study. It registers members, displays plans, calculates fees, applies discounts and generates a membership summary. It demonstrates the required Java features: Scanner, switch, if-else, operators, methods and loops. The code is modular, so plans, rates and discount rules can be changed in one place.

6.3 Limitations and Future Scope

- Data is lost when the program ends; it can be saved to a file or a database.
- Input of the wrong type (for example text instead of a number) is not handled; exception handling can be added.
- Possible extensions: member search, plan renewal, payment records, a GUI, and classes such as `Member` and `Plan` using object-oriented design.

7. References

1. Herbert Schildt, *Java: The Complete Reference*, McGraw-Hill Education.
2. E. Balagurusamy, *Programming with Java: A Primer*, McGraw-Hill Education.
3. Oracle, *The Java Tutorials*, <https://docs.oracle.com/javase/tutorial/>
4. Oracle, *Java Platform SE API Documentation: Scanner and ArrayList*, <https://docs.oracle.com/en/java/javase/>
5. GeeksforGeeks, *Java Programming Language*, <https://www.geeksforgeeks.org/java/>

